Full Name: Yeskuatov Dauren
Group: SE-2535

Design Rationale: SnowLink

Assignment 3 (Bridge + Adapter). Complexity module: dynamic implementor selection.

1. The problem

I made a small system for mountain snow stations. A station measures snow depth, temperature, wind and avalanche risk, and has to send this data out. Two things change separately. The first is the type of report: a routine report, an avalanche alert (urgent, tries again if the channel is busy) and a diagnostics report (battery level, lowest priority). The second is the way of sending: writing to a local file, sending by UDP, or sending through a satellite modem.

2. Why one pattern is not enough

With only Bridge, each report holds a channel object, so 3 reports and 3 channels need 6 classes instead of 9. But the satellite modem is from a vendor library. I cannot change it or make it implement my interface. Without an adapter, the reports would have to call the modem directly and deal with its status codes, and then they would depend on one particular channel.

With only Adapter, the modem would work with my interface, but every new report type would still need its own sending code for each channel. Bridge solves that part.

3. Why the modem does not fit

It is different in many ways, not only the method name:

	My interface (`UplinkChannel`)	**Vendor class (`SatelliteModem`)**
Method	`transmit(Frame)`	`sendBurst(byte[], int, String)`
Input	one `Frame` object	bytes, priority number, recipient name
Priority	LOW, NORMAL, CRITICAL	numbers, and 0 is the most urgent
Before sending	nothing	must call `powerUp()`, it can switch off again
Errors	`UplinkException` with a reason	int status codes, no exceptions
Size	no limit	max 340 bytes per message

`SatelliteUplinkAdapter` deals with all of this. It switches the modem on when needed, converts the frame to bytes, converts the priority, and turns every status code into my `UplinkException` with one of four reasons: `UNREACHABLE`, `OVERLOADED`, `REJECTED` or `FAULT`. The reports see only this exception, so they never know the modem exists. For example, the alert report tries again when the reason is `OVERLOADED`, and it works the same way for every channel.

4. Complexity module

I chose dynamic implementor selection. The channel is picked from a route string: `file:///...`, `udp://host:port` or `sat://name`. The client gives the string to `ChannelRegistry` and gets a channel back, so it never names a concrete class. The satellite adapter is chosen in the same way as the other two.

5. Open/Closed

To add a new report type I make a new subclass of `StationReporter`. To add a new channel I write a class and register its scheme in one line. `OpenClosedTest` shows both without changing any existing class.

6. Limitation

The modem can send only 340 bytes at once. My adapter refuses longer frames and does not split them. It is fine now because all reports are short, but a longer report would not work over satellite.